AI ASSISTED CODING
ASSIGNMENT - 13.1
ROLL NO. 2303A510C8

Task Description #1 (Refactoring – Removing Code Duplication)
• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
• Instructions:
o Prompt AI to identify duplicate logic and replace it with functions or classes.
o Ensure the refactored code maintains the same output.
o Add docstrings to all functions.
• Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

• Expected Output:
o Refactored code with a reusable function and no duplication.

Prompt:
Refactor the following Python code to remove duplication.
Identify the repeated logic and replace it with reusable functions.
Ensure the program produces the same output as the original code.
Add proper docstrings to all functions explaining their purpose and parameters.

Legacy Code:

print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))

CODE:

```
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.

    Returns:
    int or float: The area of the rectangle.
    """
    return length * width

def calculate_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.

    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.

    Returns:
    int or float: The perimeter of the rectangle.
    """
    return 2 * (length + width)

def print_rectangle_properties(length, width):
    """
    Print the area and perimeter of a rectangle.

    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.
```

```python
    """
    area = calculate_area(length, width)
    perimeter = calculate_perimeter(length, width)
    print("Area of Rectangle:", area)
    print("Perimeter of Rectangle:", perimeter)

# Rectangle dimensions
rectangles = [(5, 10), (7, 12), (10, 15)]

for length, width in rectangles:
    print_rectangle_properties(length, width)
```

OUTPUT:

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

• Task: Use AI to analyze a Python script with nested loops and complex conditionals.

• Instructions:

o Ask AI to suggest algorithmic improvements (e.g., replace nested loops with set lookups or comprehensions).

o Implement changes while keeping logic intact.

o Compare execution time before and after refactoring.

• Sample Legacy Code:

 # Legacy inefficient code

names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
print(f"{s} is not in the list")

- **Expected Output:**

o Optimized code using set lookups with performance comparison table.

Prompt:
Analyze the following Python code and refactor it to improve performance.

Instructions:

1. Identify inefficiencies caused by nested loops and complex conditionals.
2. Suggest an algorithmic improvement (for example using set lookups or list/set operations).
3. Implement the optimized version while keeping the same logic and output.
4. Measure and compare execution time between the original and optimized versions using the time module.
5. Provide a small performance comparison table showing the improvement.

CODE:

```
import time
```

```python
# Legacy Code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

# Measure execution time of the legacy code
start_time = time.time()

for s in search_names:
   found = False
   for n in names:
       if s == n:
           found = True
   if found:
       print(f"{s} is in the list")
   else:
       print(f"{s} is not in the list")

legacy_time = time.time() - start_time
print(f"Legacy code execution time: {legacy_time:.6f} seconds")

# Optimized Code
names_set = set(names)

# Measure execution time of the optimized code
start_time = time.time()

for s in search_names:
   if s in names_set:
       print(f"{s} is in the list")
   else:
       print(f"{s} is not in the list")

optimized_time = time.time() - start_time
print(f"Optimized code execution time: {optimized_time:.6f} seconds")

# Performance comparison
print("\nPerformance Comparison:")
print(f"{'Version':<20}{'Execution Time (seconds)':<30}")
print(f"{'Legacy Code':<20}{legacy_time:<30}")
print(f"{'Optimized Code':<20}{optimized_time:<30}")
```

OUTPUT:

```
Charlie is in the list
Eve is not in the list
Bob is in the list
Legacy code execution time: 0.000017 seconds
Charlie is in the list
Eve is not in the list
Bob is in the list
Optimized code execution time: 0.000004 seconds
```

Task Description #3 (Refactoring – Extracting Reusable Functions)

• Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

• Instructions:

o Identify repeated or related logic and extract it into reusable functions.

o Ensure the refactored code is modular, easy to read, and documented with docstrings.

• Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

• Expected Output:

o Code with a function calculate_total(price) that can be

reused for multiple price inputs.

Prompt:

Refactor the following Python script by extracting reusable functions.

Instructions:

1. Identify repeated calculations in the code.
2. Move the repeated logic into a reusable function named calculate_total(price).
3. Add proper docstrings explaining the function.
4. Ensure the refactored code produces the same output as the original code.
5. Make the code modular and easy to read.

Legacy Code:

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

CODE:

```
def calculate_total(price):
    """
    Calculate the total price including tax.

    Parameters:
    price (float): The initial price before tax.

    Returns:
    float: The total price after adding tax.
```

```
    """
    tax = price * 0.18
    total = price + tax
    return total

def main():
    prices = [250, 500]
    for price in prices:
        total = calculate_total(price)
        print("Total Price:", total)

if __name__ == "__main__":
    main()
```

OUTPUT:

```
ment-13.1.py
Total Price: 295.0
Total Price: 590.0
```

Task Description #4 (Refactoring – Replacing Hardcoded Values with Constants)

• Task: Use AI to identify and replace all hardcoded “magic numbers” in the code with named constants.

• Instructions:

o Create constants at the top of the file.

o Replace all hardcoded occurrences in calculations with these constants.

o Ensure the code remains functional and is easier to maintain.

• Sample Legacy Code:

 # Legacy script with hardcoded values

print("Area of Circle:", 3.14159 * (7 ** 2))

print("Circumference of Circle:", 2 * 3.14159 * 7)
• Expected Output:
o Code with constants like PI = 3.14159 and RADIUS = 7 used in calculations.

Prompt:
Refactor the following Python code by replacing all hardcoded values (magic numbers) with named constants.

Instructions:

1. Create constants at the top of the file.
2. Replace all occurrences of the hardcoded values with these constants.
3. Use descriptive constant names.
4. Ensure the code still produces the same output and is easier to maintain.

Legacy Code:
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)

CODE:

```python
# Constants
PI = 3.14159
RADIUS = 7

# Calculations
area_of_circle = PI * (RADIUS ** 2)
circumference_of_circle = 2 * PI * RADIUS

# Output
print("Area of Circle:", area_of_circle)
print("Circumference of Circle:", circumference_of_circle)
```

OUTPUT:

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Task Description #5 (Refactoring – Improving Variable Naming and Readability)

• Task: Use AI to improve readability by renaming unclear variables and adding inline comments.

• Instructions:

o Replace vague names with meaningful ones.

o Add comments where logic is not obvious.

o Keep functionality exactly the same.

• Sample Legacy Code:

```
# Legacy script with poor variable names
a = 10
b = 20
c = a * b / 2
print(c)
```

• Expected Output:

o Code with descriptive variable names like base, height, area_of_triangle, plus explanatory comments.

Prompt:

Improve the readability of the following Python code.

Instructions:

1. Replace unclear variable names with meaningful descriptive names.

2. Add inline comments explaining the logic where necessary.

3. Keep the program functionality exactly the same.
4. Ensure the final code is clean and easy to understand.

Legacy Code:
a = 10
b = 20
c = a * b / 2
print(c)

CODE:

```
# Define the length and width of a rectangle
length = 10
width = 20

# Calculate the area of the rectangle
area = length * width / 2

# Print the calculated area
print(area)
```

OUTPUT:

```
ment-13.1.py
100.0
```

Task Description #6 (Refactoring – Removing Redundant Conditional Logic)
• Task: Use AI to refactor a Python script that contains repeated if–else
logic for grading students.
• Instructions:
o Ask AI to identify redundant conditional checks.
o Replace them with a reusable function.

o Ensure grading logic remains unchanged.
• Code:

```
marks = 85
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

• Expected Output:
o A reusable function like calculate_grade(marks) with clean logic and docstring.

Prompt:
Refactor the following Python code to remove redundant conditional logic.

Instructions:

1. Identify repeated grading logic.
2. Create a reusable function called calculate_grade(marks).
3. Move the grading conditions into the function.
4. Add a docstring explaining the function.
5. Ensure the grading output remains exactly the same.

Legacy Code:

```
marks = 85
if marks >= 90:
```

print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")

marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")

CODE:

```
def calculate_grade(marks):
    """
    Calculate the grade based on the given marks.

    Args:
        marks (int): The marks obtained by the student.

    Returns:
        str: The grade corresponding to the marks.
    """
    if marks >= 90:
        return "Grade A"
    elif marks >= 75:
        return "Grade B"
    else:
        return "Grade C"

# Example usage
marks_list = [85, 72]
for marks in marks_list:
    print(calculate_grade(marks))
```

OUTPUT:

```
Grade B
Grade C
```

Task Description #7 (Refactoring – Converting Procedural Code to Functions)
• Task: Use AI to refactor procedural input–processing logic into functions.
• Instructions:
o Identify input, processing, and output sections.
o Convert each into a separate function.
o Improve code readability without changing behavior.
• Sample Legacy Code:
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
• Expected Output:
o Modular code using functions like get_input(), calculate_square(), and display_result().

Prompt:
Refactor the following procedural Python code into modular functions.

Instructions:

1. Identify the input, processing, and output parts of the program.
2. Create separate functions:

*** get_input()**

* calculate_square()
* display_result()

3. Ensure the behavior of the program remains the same.
4. Add docstrings to each function.
5. Improve readability and modularity.

Legacy Code:

```
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```

CODE:

```python
def get_input():
    """Get user input and return it as an integer."""
    return int(input("Enter number: "))

def calculate_square(num):
    """Calculate the square of a given number."""
    return num * num

def display_result(square):
    """Display the square of the number."""
    print("Square:", square)

def main():
    """Main function to execute the program."""
    num = get_input()
    square = calculate_square(num)
    display_result(square)

if __name__ == "__main__":
    main()
```

OUTPUT:

```
Enter number: 7
```

```
Square: 49
```

Task Description #8 (Refactoring – Optimizing List Processing)
• Task: Use AI to refactor inefficient list processing logic.
• Instructions:
o Ask AI to replace manual loops with list comprehensions or built-in functions.
o Ensure output remains identical.
• Sample Legacy Code:

```
nums = [1, 2, 3, 4, 5]
squares = []
for n in nums:
squares.append(n * n)
print(squares)
```

• Expected Output:
o Optimized version using list comprehension with improved readability

Prompt:
Refactor the following Python code to optimize list processing.

Instructions:

1. Identify inefficient manual loops.
2. Replace them with Python list comprehensions or built-in functions.
3. Ensure the output remains identical.
4. Improve readability and conciseness of the code.

Legacy Code:

```
nums = [1, 2, 3, 4, 5]
squares = []
```

for n in nums:
squares.append(n * n)

print(squares)

CODE:

```
nums = [1, 2, 3, 4, 5]
squares = [n * n for n in nums]

print(squares)
```

OUTPUT:

```
[1, 4, 9, 16, 25]
```